Typer

Typer, build great CLIs. Easy to code. Based on Python type hints.

Test passing | Publish passing | Coverage | pypi package v0.21.1

Documentation: https://typer.tiangolo.com

Source Code: https://github.com/fastapi/typer

Typer is a library for building CLI applications that users will **love using** and developers will **love creating**. Based on Python type hints.

It's also a command line tool to run scripts, automatically converting them to CLI applications.

The key features are:

- **Intuitive to write**: Great editor support. Completion everywhere. Less time debugging. Designed to be easy to use and learn. Less time reading docs.
- **Easy to use**: It's easy to use for the final users. Automatic help, and automatic completion for all shells.
- **Short**: Minimize code duplication. Multiple features from each parameter declaration. Fewer bugs.
- **Start simple**: The simplest example adds only 2 lines of code to your app: **1 import, 1 function call**.

- **Grow large**: Grow in complexity as much as you want, create arbitrarily complex trees of commands and groups of subcommands, with options and arguments.
- **Run scripts**: Typer includes a `typer` command/program that you can use to run scripts, automatically converting them to CLIs, even if they don't use Typer internally.

2026 February - Typer developer survey

Help us define Typer's future by filling the Typer developer survey [🔄]. ✨

FastAPI of CLIs

Typer is FastAPI [🔄]'s little sibling, it's the FastAPI of CLIs.

Installation

Create and activate a virtual environment [🔄] and then install **Typer**:

```bash
```

Example

The absolute minimum

- Create a file `main.py` with:

```python
def main(name: str):
    print(f"Hello {name}")
```

This script doesn't even use Typer internally. But you can use the `typer` command to run it as a CLI application.

Run it

Run your application with the `typer` command:

```bash
```

This is the simplest use case, not even using Typer internally, but it can already be quite useful for simple scripts.

Note: auto-completion works when you create a Python package and run it with `--install-completion` or when you use the `typer` command.

Use Typer in your code

Now let's start using Typer in your own code, update `main.py` with:

```python
import typer


def main(name: str):
    print(f"Hello {name}")


if __name__ == "__main__":
    typer.run(main)
```

Now you could run it with Python directly:

```bash
```

Note: you can also call this same script with the `typer` command, but you don't need to.

Example upgrade

This was the simplest example possible.

Now let's see one a bit more complex.

An example with two subcommands

Modify the file `main.py`.

Create a `typer.Typer()` app, and create two subcommands with their parameters.

```
import typer

app = typer.Typer()


@app.command()
def hello(name: str):
    print(f"Hello {name}")


@app.command()
```

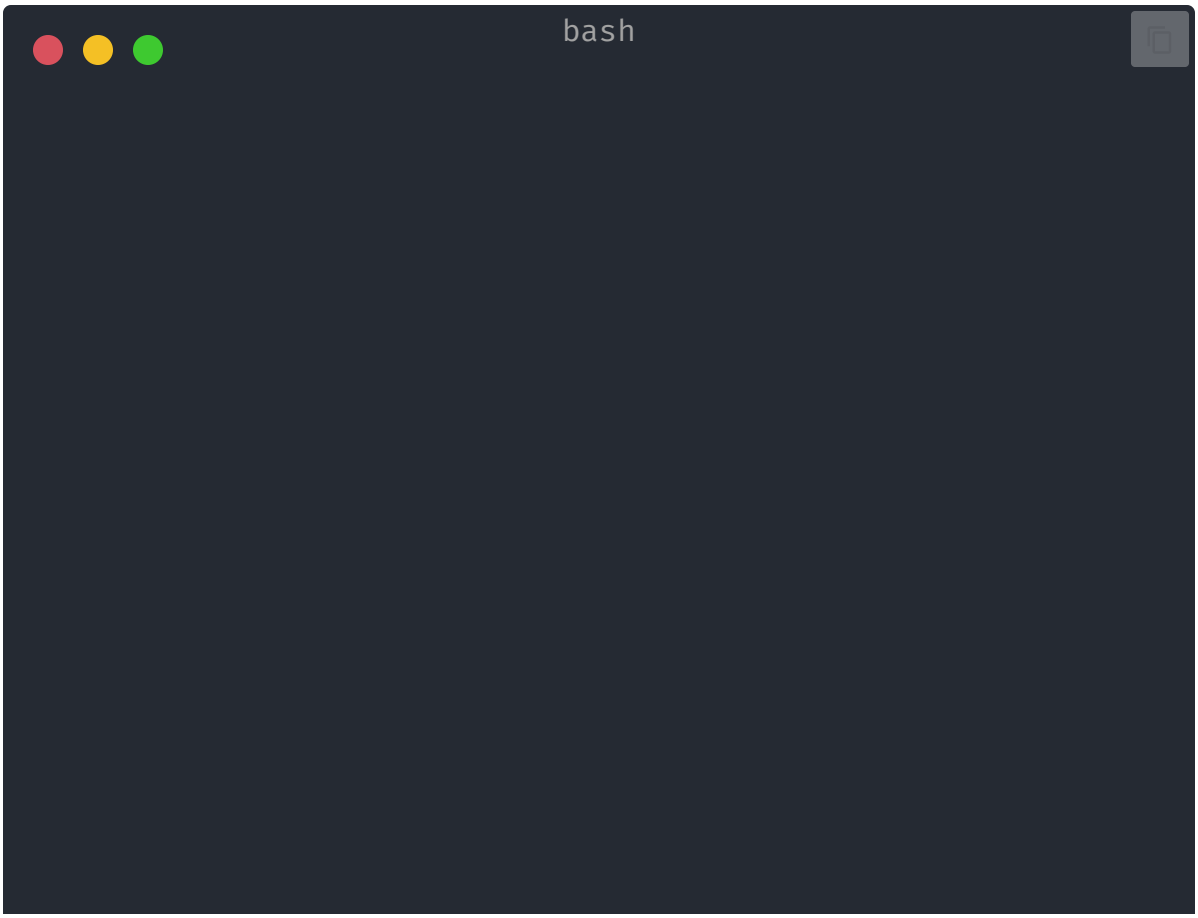
```python
def goodbye(name: str, formal: bool = False):
    if formal:
        print(f"Goodbye Ms. {name}. Have a good day.")
    else:
        print(f"Bye {name}!")


if __name__ == "__main__":
    app()
```

And that will:

- Explicitly create a `typer.Typer` app.
  - The previous `typer.run` actually creates one implicitly for you.
- Add two subcommands with `@app.command()`.
- Execute the `app()` itself, as if it was a function (instead of `typer.run`).

Run the upgraded example

Check the new help:



Now check the help for the `hello` command:

```bash
```

And now check the help for the `goodbye` command:

```bash
```

Now you can try out the new command line application:

bash

Note: If your app only has one command, by default the command name is **omitted** in usage: `python main.py Camila`. However, when there are multiple commands, you must **explicitly**

include the command name: `python main.py hello Camila`. See One or Multiple Commands for more details.

Recap

In summary, you declare **once** the types of parameters (*CLI arguments* and *CLI options*) as function parameters.

You do that with standard modern Python types.

You don't have to learn a new syntax, the methods or classes of a specific library, etc.

Just standard **Python**.

For example, for an `int`:

```
total: int
```

or for a `bool` flag:

```
force: bool
```

And similarly for **files**, **paths**, **enums** (choices), etc. And there are tools to create **groups of subcommands**, add metadata, extra **validation**, etc.

You get: great editor support, including **completion** and **type checks** everywhere.

Your users get: automatic `--help`, **auto-completion** in their terminal (Bash, Zsh, Fish, PowerShell) when they install your package or when using the `typer` command.

For a more complete example including more features, see the Tutorial - User Guide.

Dependencies

Typer stands on the shoulders of a giant. Its only internal required dependency is Click [↪].

By default it also comes with extra standard dependencies:

- `rich` [↪]: to show nicely formatted errors automatically.
- `shellingham` [↪]: to automatically detect the current shell when installing completion.
  - With `shellingham` you can just use `--install-completion`.

- Without `shellingham`, you have to pass the name of the shell to install completion for, e.g. `--install-completion bash`.

`typer-slim`

If you don't want the extra standard optional dependencies, install `typer-slim` instead.

When you install with:

```
pip install typer
```

...it includes the same code and dependencies as:

```
pip install "typer-slim[standard]"
```

The `standard` extra dependencies are `rich` and `shellingham`.

Note: The `typer` command is only included in the `typer` package.

License

This project is licensed under the terms of the MIT license.